CMPSC 311 - Introduction to Systems Programming

Dynamic Memory

“You are now all sanitation engineers”

Memory allocation

PennState

- So far, we have seen two kinds of memory allocation:

```
// a global variable
int counter = 0;

int main(int argc, char **argv)
{
  counter++;
  return 0;
}
```

```
int foo(int a) {
  int x = a + 1;  // local var
  return x;
}

int main(int argc, char **argv) {
  int y = foo(10);  // local var
  return 0;
}
```

`counter` is *statically* allocated

- allocated when program is loaded
- deallocated when program exits
- Memory in the *data/bss/text* segments

`a,x,y` are *automatically/temporarily* allocated

- allocated when function is called
- deallocated when function returns
- Memory in the *stack* segment

We need more flexibility

PennState

- Sometimes we want to allocate memory that:
  - is allocated at runtime **(not statically)**
  - but persists across multiple function calls **(not automatic/temporary)**
    - *less than the lifetime of the program, longer than the lifetime of a function*

- We want to:
  - Handle data too big to fit on the stack
  - Handle data where its size is not known in advance
  - Allocate the memory ourselves

- **This is called *dynamic* memory**

```
// (this is pseudo-C-code)
char* read_file(char* filename) {
  int   size = get_file_size(filename);
  char* buffer = AllocateMemory(size);
  read_file_into_buffer(filename, buffer);
  return buffer;
}
```

Dynamic Memory Allocation

PennState

- What we want is *dynamically allocated memory*
  - We want to explicitly request a new block of memory
  - But how?
    - The language runtime allocates it, perhaps with help from OS
  - Dynamically allocated memory persists until:
    - your code explicitly deallocates it *[manual memory management]*
      - *C, C++, Rust*
    - a garbage collector collects it *[automatic memory management]*
      - *Java, Python, Go*
  - C requires you to <u>manually manage dynamic memory</u>
    - gives you more control and performance, but can cause headaches
    - <u>*C has no garbage collection.*</u> <u>**YOU ARE THE GARBAGE COLLECTOR**</u>

Aside: Garbage Collection, in general

- In some languages like Java, you can dynamically allocate objects using the built-in "`new`" function of the Java runtime environment

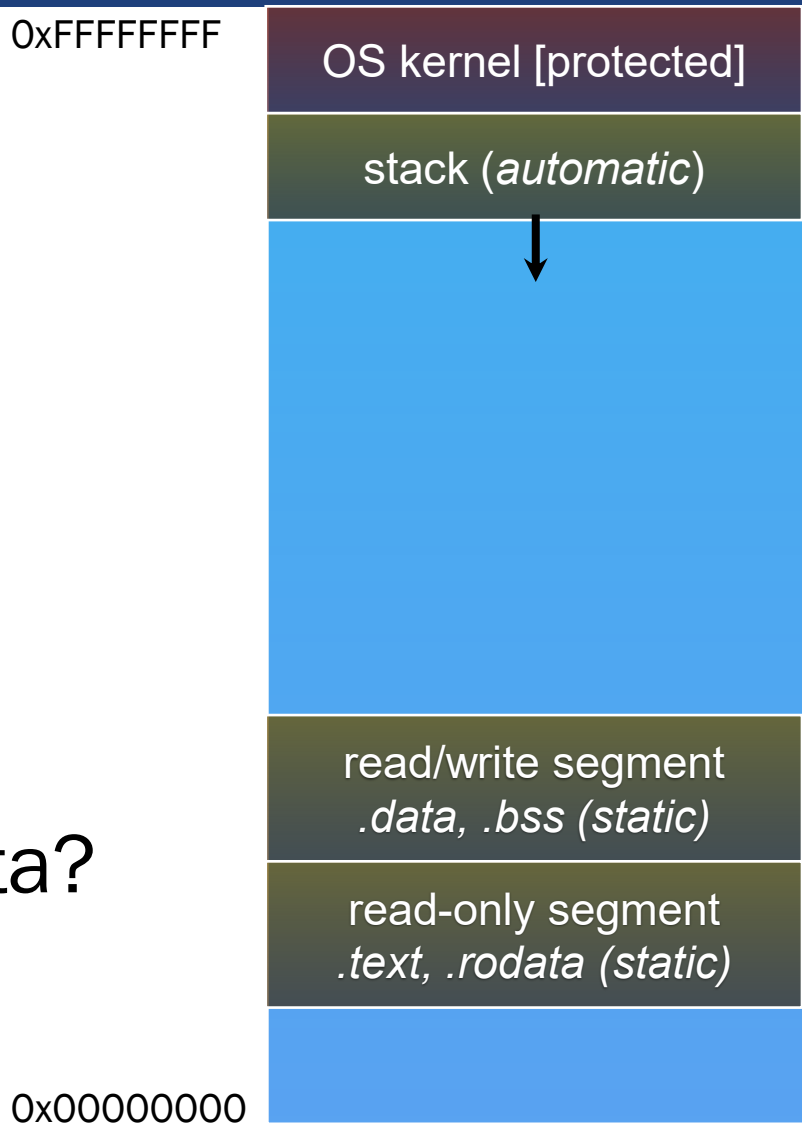
```
String str1 = new String("This is a text string");
```

- Stays in memory until an invisible process behind the scenes frees.
  - This invisible process is called *garbage collection*
  - Typically done in the background by a background process
  - Tracks scope, use, reference, and other metadata to know when to reap objects
- Pros: a large class of memory bugs avoided
  - dereferencing dangling pointers, memory leaks, double freeing of memory, …
- Cons: unpredictable performance
  - garbage collection can start at an arbitrary time and slow down the program*

Dynamic Memory Allocation

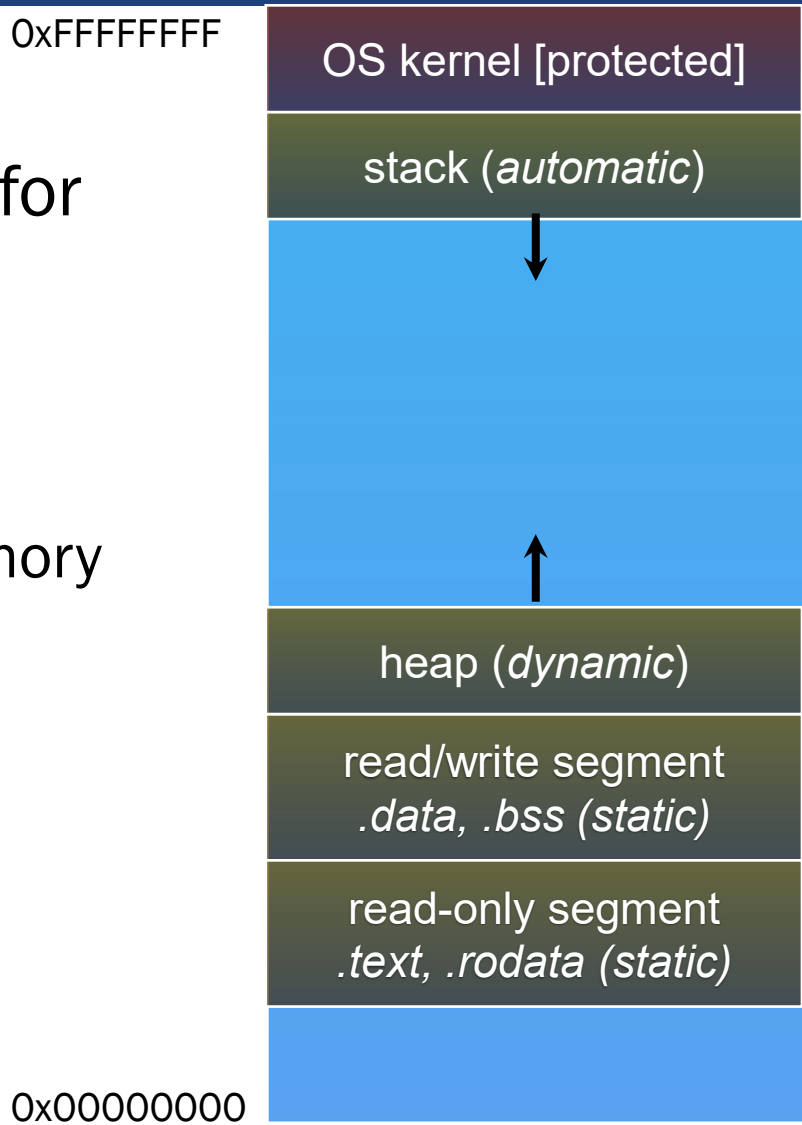
- Remember, we want dynamic memory to allow us to:
  - … allocate an arbitrary number of bytes
  - … use and release (or “free”) memory at any point
    - Non-static, non-automatic
  - … not interfere with our existing segments

- Given these wants, where do we place said data?



The Heap

- The heap is a segment in virtual memory that:
  - holds a large ***pool*** of unused memory that is used for dynamically allocated data
  - Unordered…
    - Allocating new memory doesn't necessarily mean new memory will be "above" or "before" other dynamic memory
  - …but structured (usually)
    - Most libraries inject hidden metadata about how big a dynamic allocation is (473 Topic)
  - Grows from low addresses to high addresses

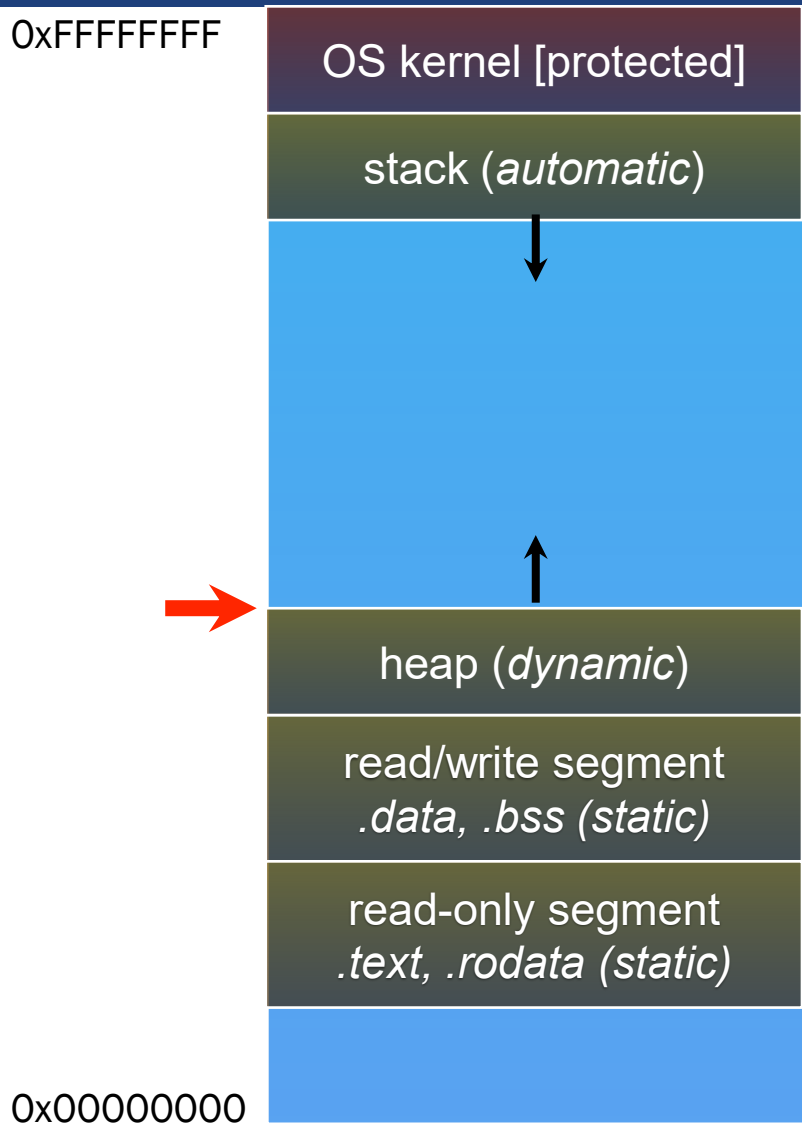


The heap, at a glance

- Ultimately the **kernel** is responsible for allocating memory to a process
  - Every program begins with a certain amount of memory in its heap

- The top of the heap is known and the *program break*.

- The program break gets moved up and down as memory is allocated and deallocated from the heap.



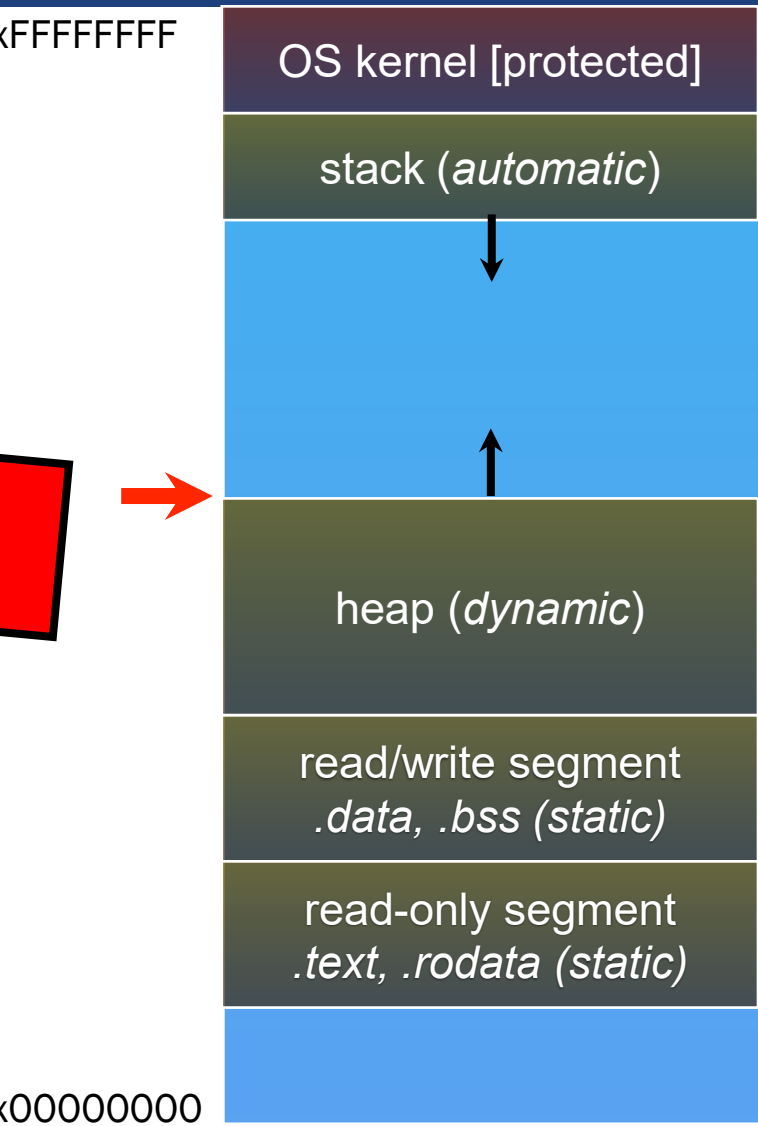
The heap, at a glance

- Ultimately the kernel is responsible for allocating memory to a process
  - Every program begins with a certain amount of memory in its heap

- The top of the heap is known and the *program break*.

- The program break gets moved up and down as memory is allocated and deallocated from the heap.

So then how do we tell the kernel to change this?



brk() and sbrk() System Calls

PennState

- The functions are system calls to request a change in program break
  - `void* brk( void* ptr )` – requests the program break to be set at the address `ptr`
    - This is an absolute pointer, so very dangerous
    - Calling `brk(NULL)` returns the current program break
  - `void* sbrk( int inc )` – requests the program break to increment `inc` bytes upwards or downwards
    - A positive `inc` allocates new memory
    - A negative `inc` frees memory

These functions are declared in the system library **`unistd.h`**

Are we going to use these calls?

- In short, no.
  - In this class we will not directly call these system calls, we will use a system library
- In 473, yes.
  - You will implement your own dynamic memory allocation library which directly uses these system calls
- Then why tell us?
  - The libraries we will use make these calls for us; they are wrappers / abstractions.
  - Important to understand that you are <u>still making system calls</u>!

Dynamic Memory Allocation in C

- So how do we actually request memory dynamically from our operating system in C?
  - Using the following system library in ***stdlib.h***

```
void* malloc(size_t size);
void* calloc(size_t nmemb, size_t size);
void* realloc(void* ptr, size_t size)
void  free(void* ptr);
```

C and malloc

PennState

- `malloc` allocates a block of memory of the given size ***in bytes***

```
void* malloc(size_t bytes)
```

  - returns a pointer to the first byte of raw memory (returns a `void*`)
    - It's best practice to cast the return for readability
    - Though it is not really necessary
      - void pointers are automatically promoted/casted

- `malloc` returns `NULL` if the memory could not be allocated (very rare)
- You'll typically use `sizeof()` to calculate the size you need

```
// allocate a single float
float* my_float = (float*)malloc(sizeof(float));
if (my_float == NULL) return -1;
*my_float = 3.14159;
```

```
// allocate a 10 element int array
int* arr = (int*)malloc(10 * sizeof(int));
if (arr == NULL) return -1;
arr[0] = 5;
arr[3] = 123;
```

Dynamically allocated types

- You can `malloc()` any type; primitive, struct, etc…
  - Allows you to make functions that allocate memory / create objects for you!

```
typedef struct {
  double real;  // real component
  double imag;  // imaginary component
} complex_t,

complex_t* create_complex(double real, double imag) {
  complex_t* newcomp = (complex_t*)malloc(sizeof(complex_t));
  if (newcomp == NULL) return NULL;
  newcomp->real = real;
  newcomp->imag = imag;
  return newcomp;
}

void main(void) {
  complex_t* sqneg1 = create_complex(0, 1);
  sqneg1->real == 0; // evals true
  sqneg1->imag == 1; // evals true
}
```

But didn't we say not to do this?

- Okay to return a pointer to memory that is not managed by your scope

```
int* max(int* a, int* b) {
  return (*a > *b) ? a : b;
}

void main(void) {
  int x = 5, y = 6;
  printf("max of %d and %d is %d\n", x, y, *max(&x, &y));
}
```

- NOT OKAY TO RETURN A POINTER TO A LOCAL VARIABL WHY?)

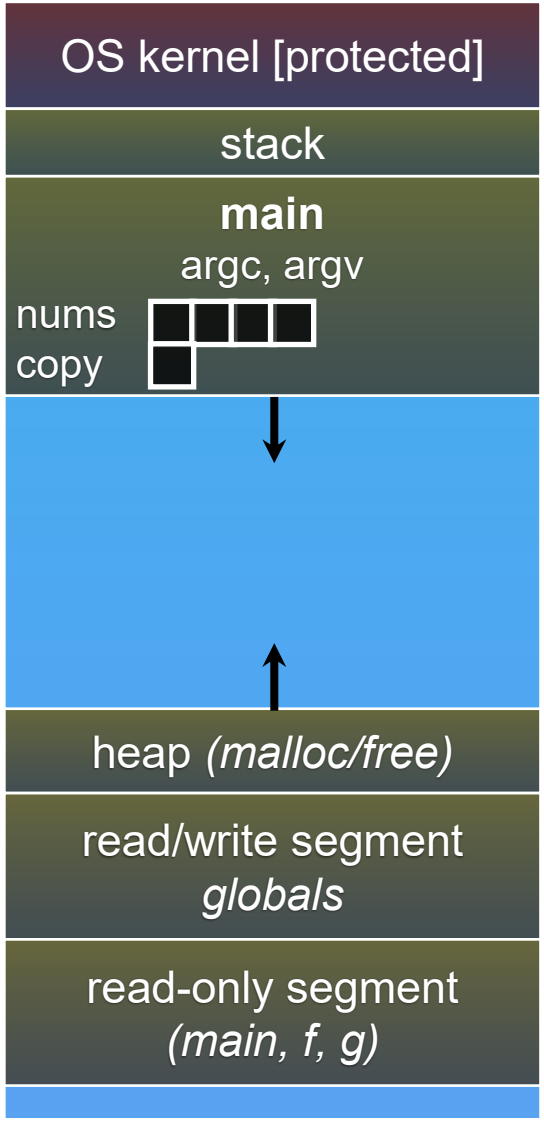
```
int* foo(void) {
  int x = 5;
  printf("%d\n", *foo());
}
```

REMEMBER: MALLOC MEMORY IS NOT STACK ALLOCATED, ITS ON THE HEAP (NOT YOUR "SCOPE")!

Dynamic Memory in action

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);

  return 0;
}
```

arraycopy.c

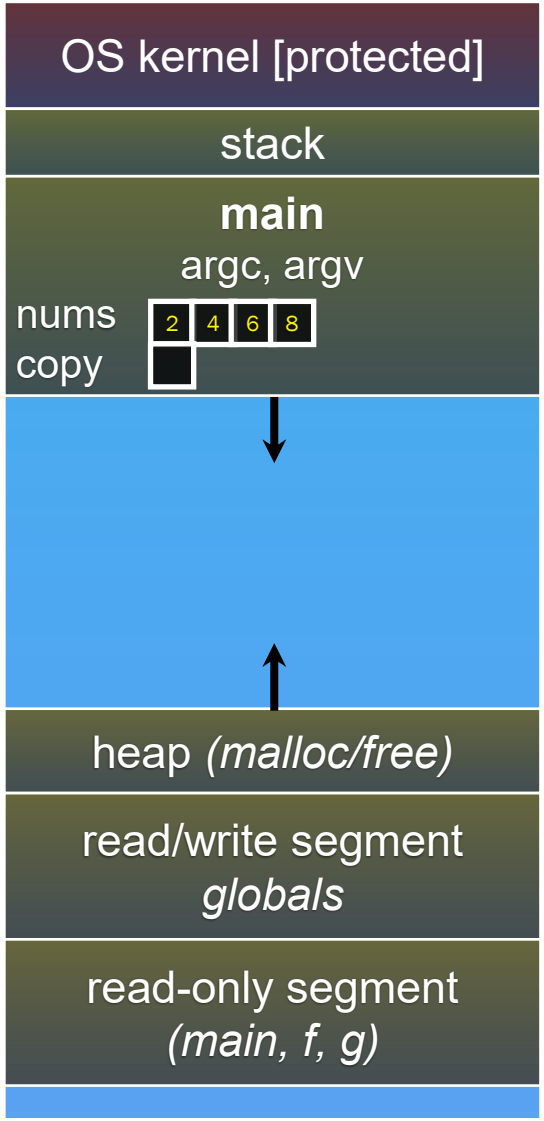


Dynamic Memory in action

PennState

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);

  return 0;
}
```

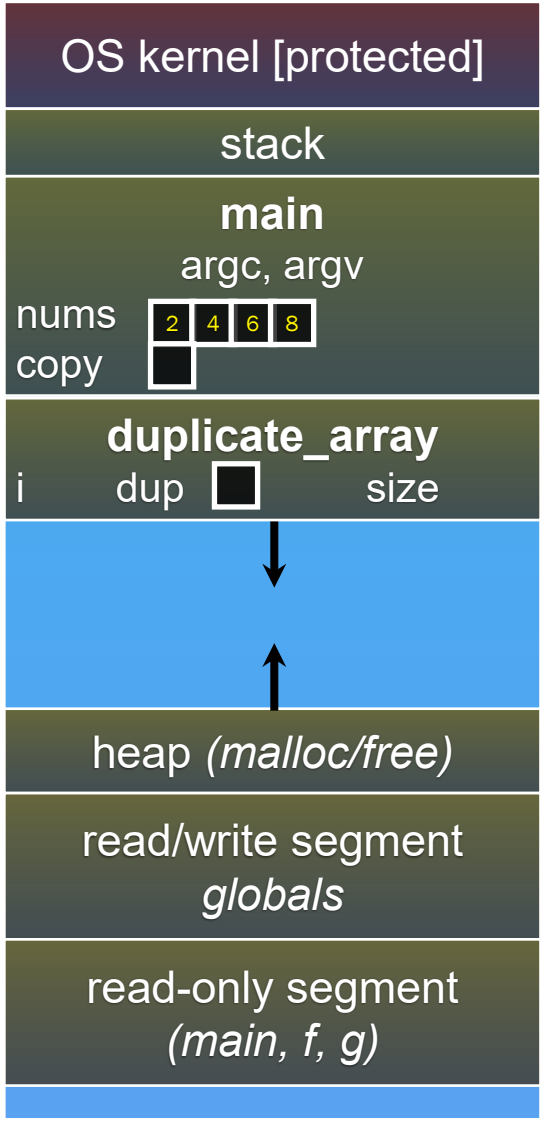
arraycopy.c



Dynamic Memory in action

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);

  return 0;
}
```

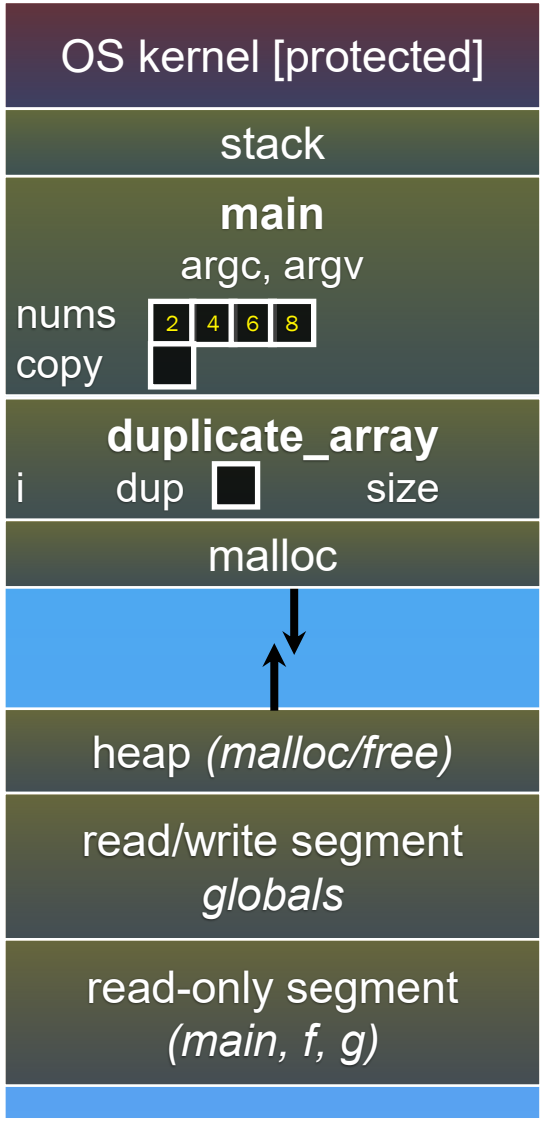
arraycopy.c



Dynamic Memory in action

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);

  return 0;
}
```

arraycopy.c

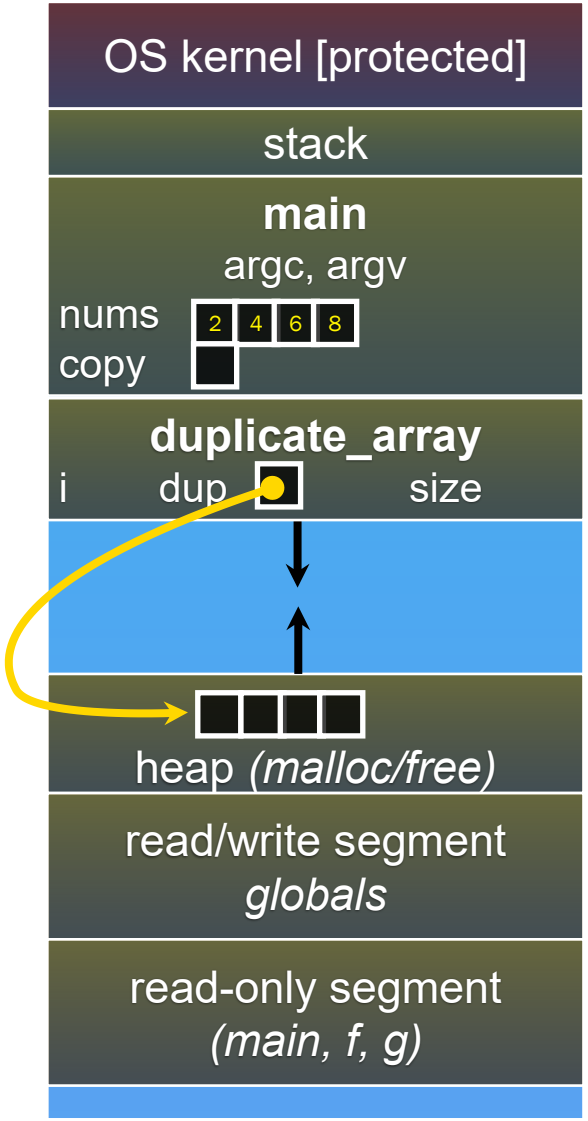


Dynamic Memory in action

PennState

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);

  return 0;
}
```

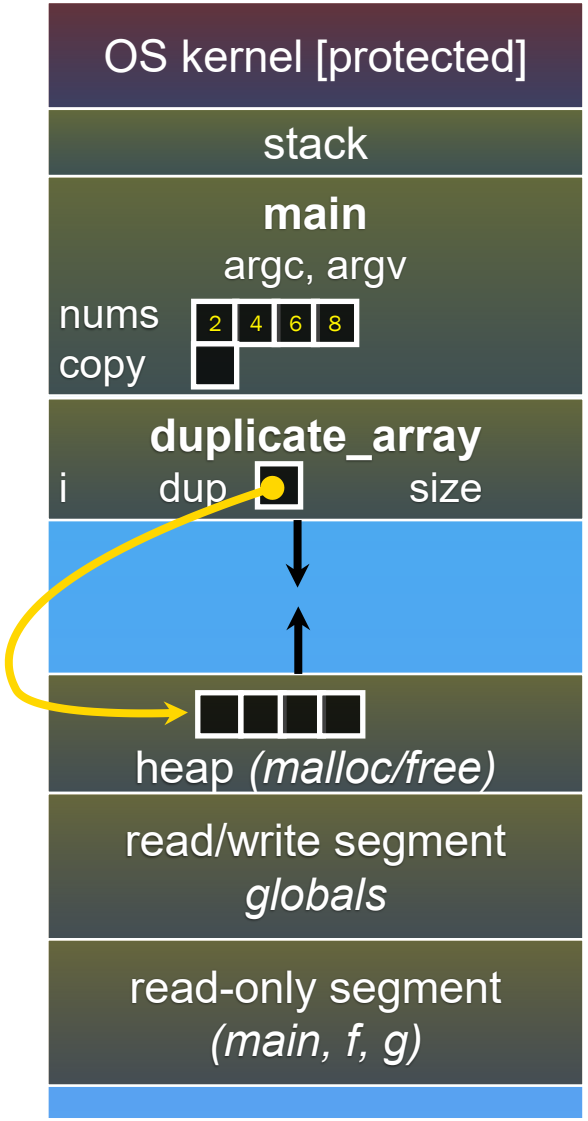
arraycopy.c



Dynamic Memory in action

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);

  return 0;
}
```

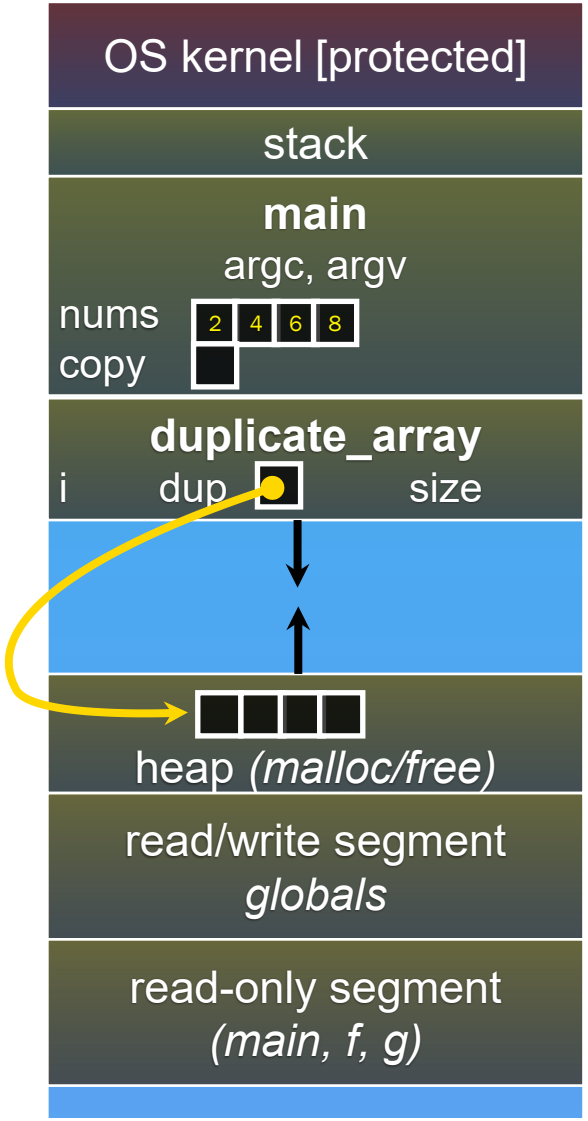
arraycopy.c



Dynamic Memory in action

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);

  return 0;
}
```

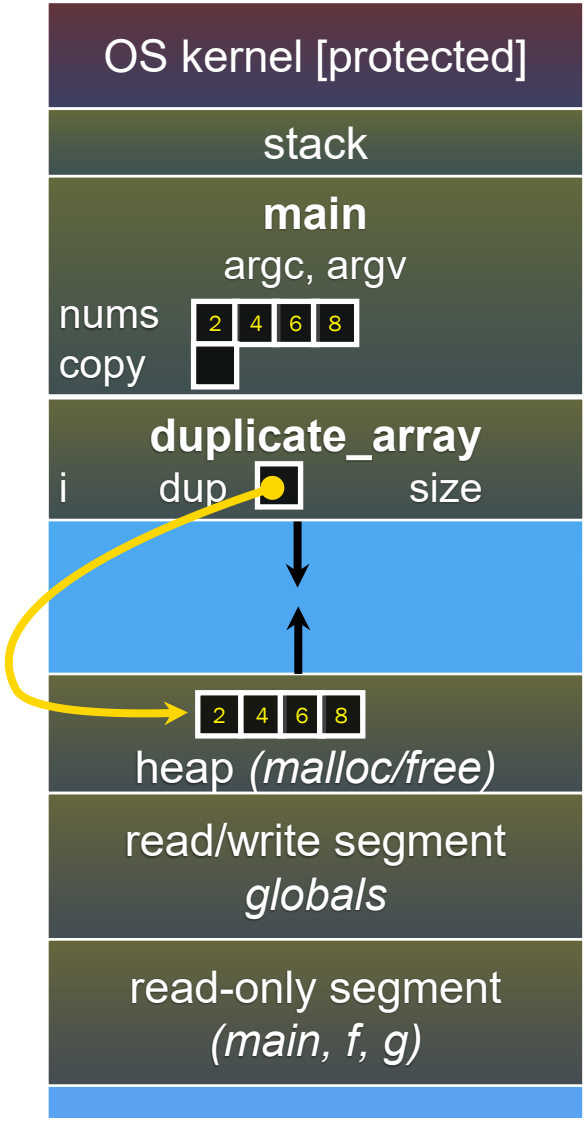
arraycopy.c



Dynamic Memory in action

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);

  return 0;
}
```

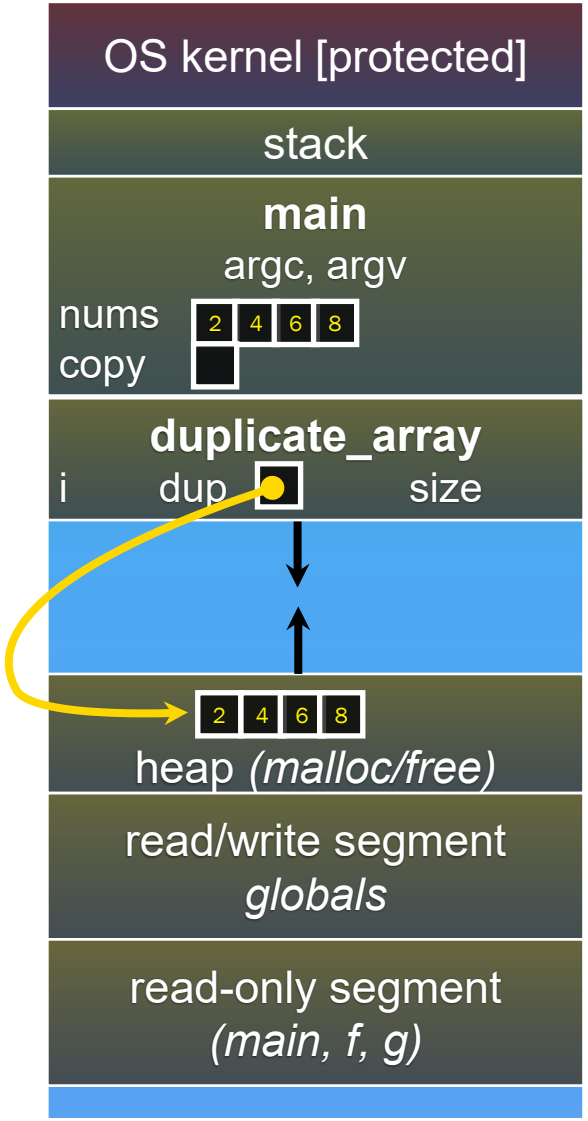
arraycopy.c



Dynamic Memory in action

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);

  return 0;
}
```

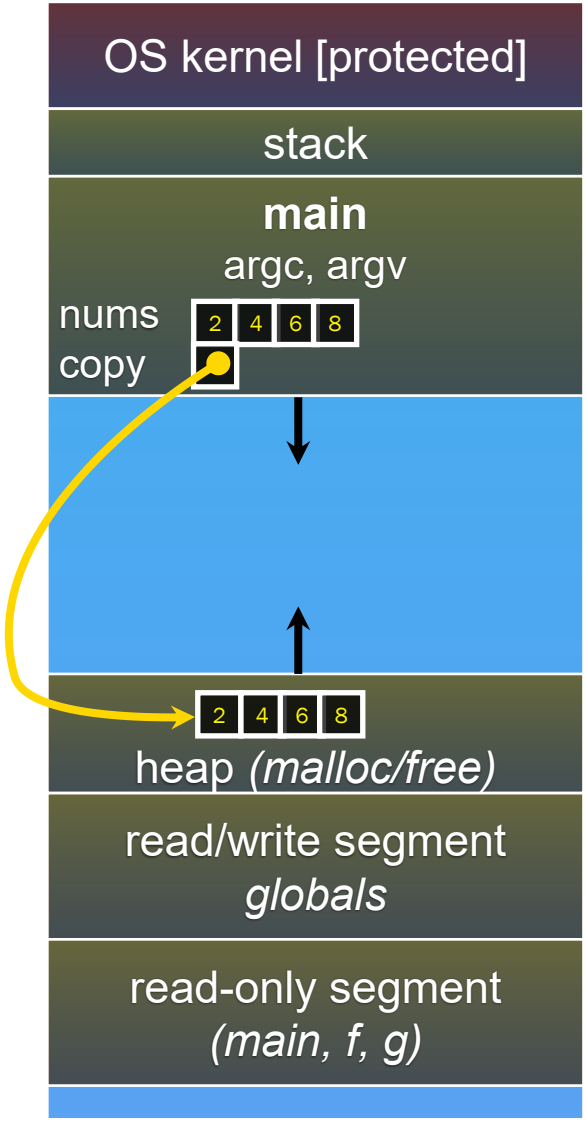
arraycopy.c



Dynamic Memory in action

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);

  return 0;
}
```

arraycopy.c



C and calloc

- When you use malloc, the memory is **uninitialized**, ***contains garbage***

- If you want to zero-out the memory, you can alternatively use calloc

```
void* calloc(size_t nmemb, size_t bytes);
```

  - Returns an array of `nmemb` members, each of size `bytes`
  - Memory is zeroed out (all bytes have the value 0x0)
    - Performance hit, but preferred for non-performance-critical code

```
// allocate a 10 element int array initialized to 0
int* arr = (int*)calloc(10, sizeof(int));
if (arr == NULL) return -1;
arr[0..9] == 0; // all true
```

Memory Operations

- Given that malloc and calloc return raw memory pointers, it is useful to treat these regions as byte-element buffers (when left uncast)

- Given two raw memory addresses, we can perform byte-wise memory operations:
  - Initialize memory byte-wise to something (set all buffer bytes to be 0x3)
  - Compare memory byte-wise (is buffer 1 == buffer 2)
  - Copy memory byte-wise (set buffer 1 = buffer 2)

  - *Some of this should conceptually be similar to what you've already done in homework / used in other standard libraries (such as string.h)*

Filling memory

- `memset` fills `n` bytes of `buf` memory with a given constant byte `value`

```
void* memset(void* buf, unsigned char value, size_t n);
```

Returns the buffer that was passed in.

```
char a[4] = {0, 1, 2, 3};
memset(a, 0, 4);
// what are the contents of a now?

int b[4] = {0, 1, 2, 3};
memset(b, 10, 4);
// what are the contents of b now? Be careful!
```

Comparing memory

- We often want to compare buffers to see if they match
  - And sometimes also to see if one is byte-wise smaller or larger

- `memcmp` compares first `n` bytes of buffers

```
int memcmp(void* buf1, void* buf2, size_t n);
```

- The comparison function returns:
  - A negative integer if buf1 is less than buf2
  - Zero if buf1 is equal to buf2
  - A positive integer is buf1 greater than buf2

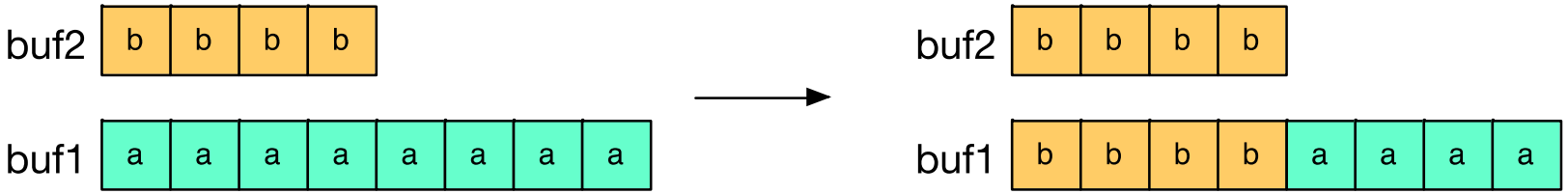
```
memcpy(dest, src, n)
is kinda like
dest = src
```

Copying memory

- `memcpy` copies one memory region to another byte-wise
  - Copy `n` bytes from `src` ("source") buffer to `dst` ("destination") buffer

```
void* memcpy(void* dst, void* src, size_t n);
```

```
char buf1[8] = { 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a' };
char buf2[4] = { 'b', 'b', 'b', 'b' };
memcpy(buf1, buf2, 4);
```



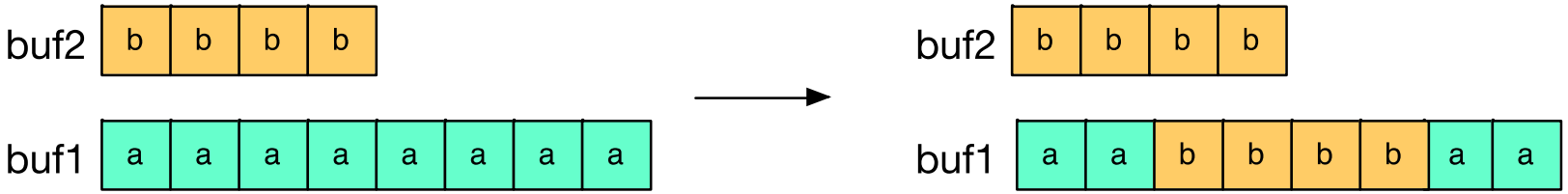
Copying memory

- `memcpy` copies one memory region to another
  - Copy `n` bytes from `src` ("source") buffer to `dst` ("destination") buffer

```
void* memcpy(void* dst, void* src, size_t n);
```

```
char buf1[8] = { 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a' };
char buf2[4] = { 'b', 'b', 'b', 'b' };
memcpy(buf1 + 2, buf2, 4);
```

Buffer splicing with pointer arithmetic



Realloc (re-allocation)

- `realloc` changes a previous allocation (resizes it)

```
void *realloc(void *ptr, size_t size)
```

- Resizes the previous allocation in place, *if possible*
  - Can be resized larger, or smaller!
- If it can't resize in place, it will create a new allocation and copy data for you
  - Essentially does:
    - new_allocation = malloc(new_size)
    - memcpy(new_allocation, old_allocation, new_size)
- returns `NULL` if the memory could not be allocated

Back to dynamic memory

- We have looked at allocating memory dynamically with malloc, calloc, realloc
  - And alternatively using some memory functions to perform operations on such

- But now how do we tell the operating system we are done with said memory?
  - And what would happen if we never did?

My C program exit
without freeing allocated memory

OS:

Dynamic Memory Deallocation

PennState

- To reap/release/free any dynamically allocated data, use **free**

```
void free(void* ptr);
```

  - The passed pointer must point to the **first byte** of **heap-allocated** memory
    - i.e., what previously returned by a `malloc( )` or `calloc( )`
    - ***Free does NOT work on statically or automatically allocated memory!****
    - **free(NULL) does nothing (for safety, we'll talk about it)**

- After freeing a block of memory, that block of memory can be used by some future `malloc/calloc`
  - **Enables memory reuse!**

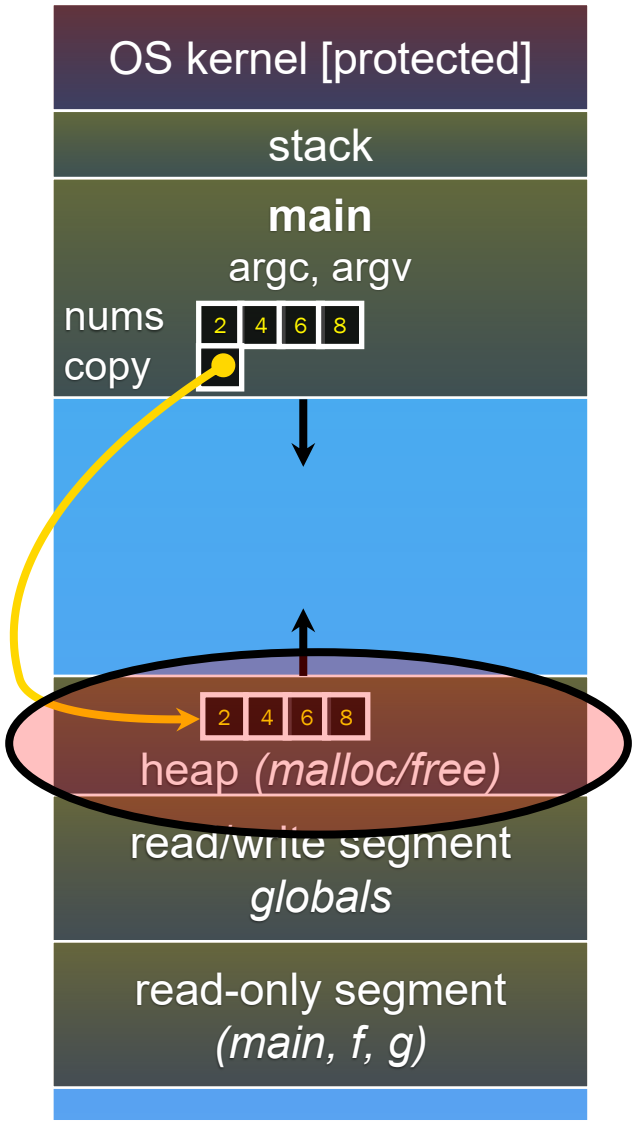
```
// allocate a 10 element int array
int* arr = (int*)malloc(10 * sizeof(int));
if (arr == NULL) return -1;
arr[0] = 5;
// some more work / functions / returns...
// eventually when done...
free(arr);
```

Dynamic Memory in action

PennState

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);

  return 0;
}
```

arraycopy.c

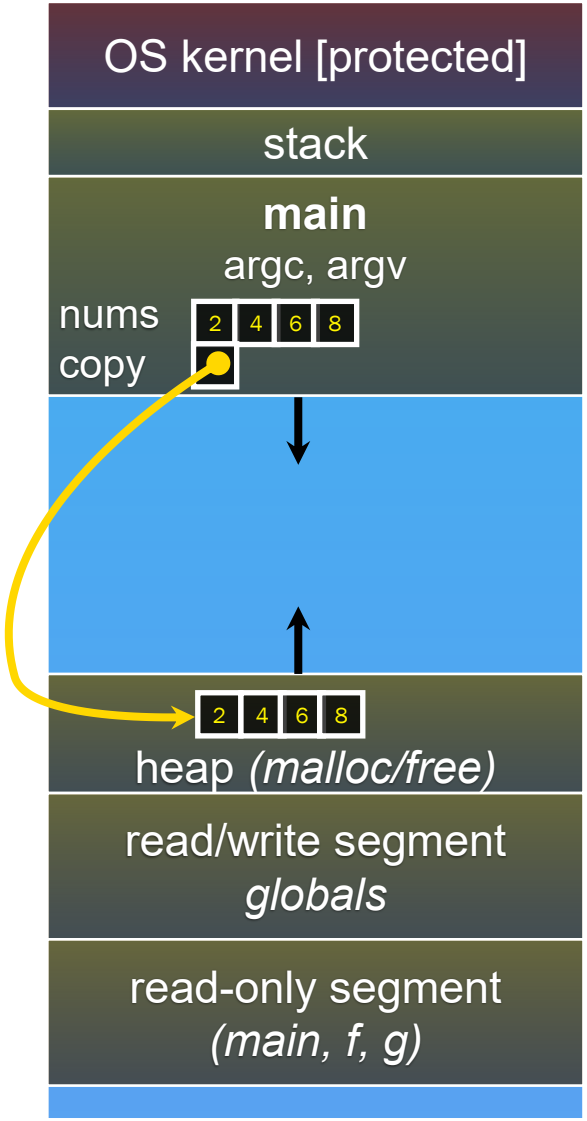


Dynamic Memory in action

PennState

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);
  free(copy);
  return 0;
}
```

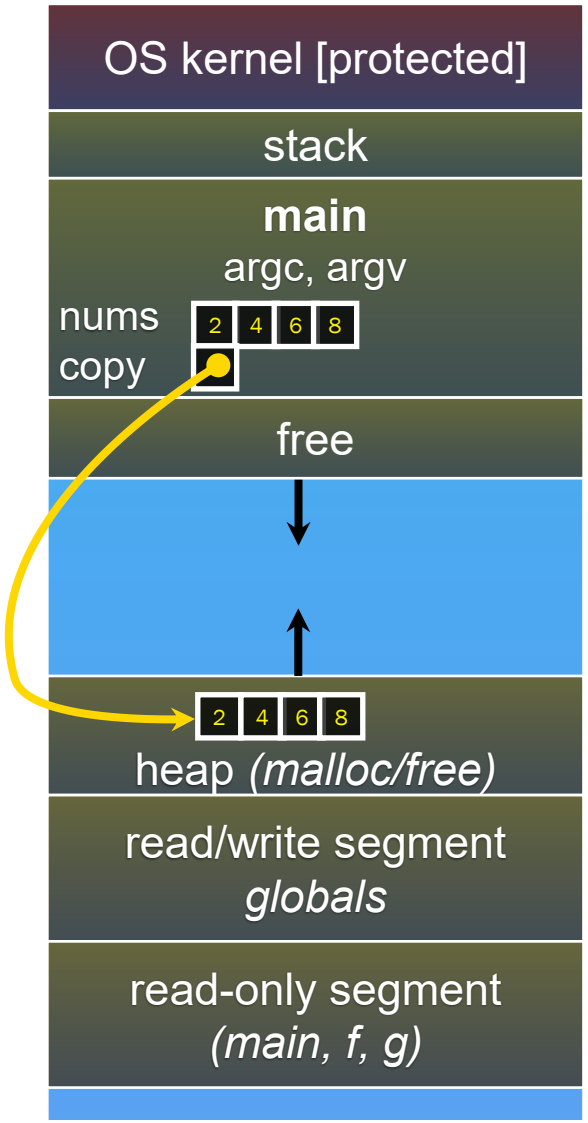
arraycopy.c



Dynamic Memory in action

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);
  free(copy);
  return 0;
}
```

arraycopy.c

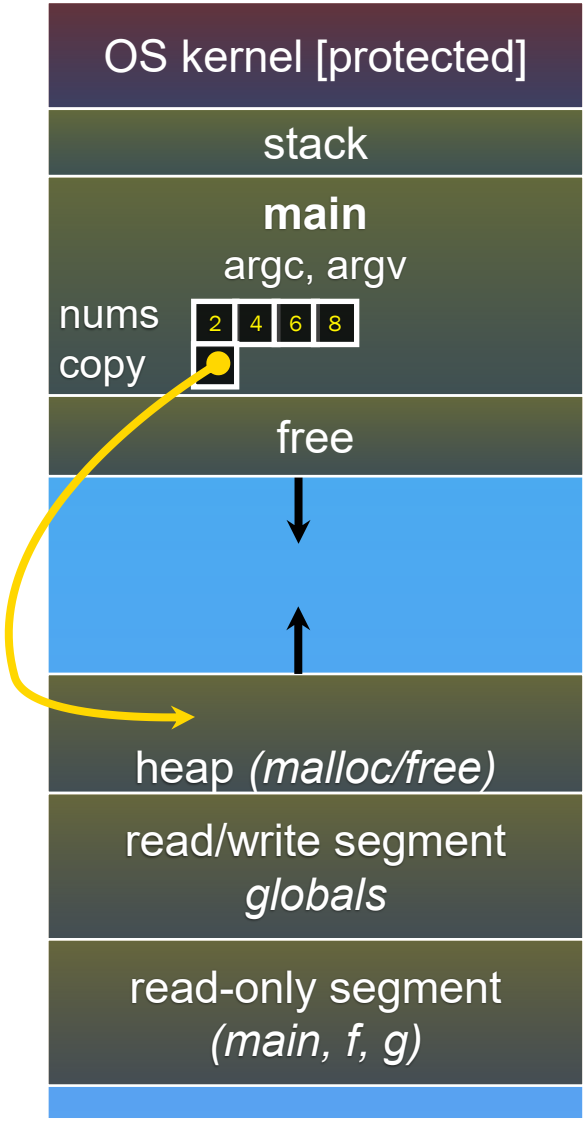


Dynamic Memory in action

PennState

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);
  free(copy);
  return 0;
}
```

arraycopy.c

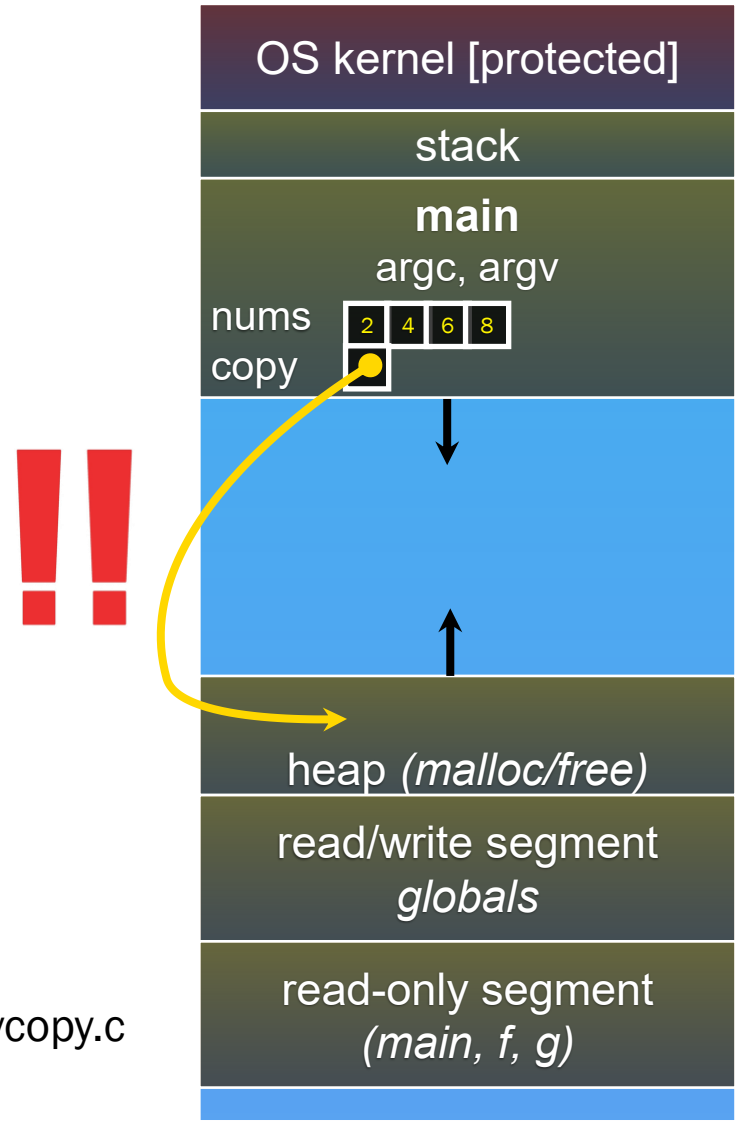


Dynamic Memory in action

PennState

```c
#include <stdlib.h>

int* duplicate_array(int* arr, int size) {

  int* dup = malloc(size * sizeof(int));

  if (dup == NULL) return NULL;

  for (int i = 0; i < size; i++)
    dup[i] = arr[i];

  return dup;
}

void main(int argc, char** argv) {
  int nums[4] = {2, 4, 6, 8};
  int* copy = duplicate_array(nums, 4);
  free(copy);
  return 0;
}
```

arraycopy.c



Dangling Pointers

- When a pointer points to a memory address that is accessible (non-NULL) but not valid (unallocated/maintained by someone else) it is said that the pointer is a **dangling pointer**

- Why is this an issue?
  - What would happen if you started to use a pointer after it has been freed?
  - It could have been allocated to some other dynamic call!

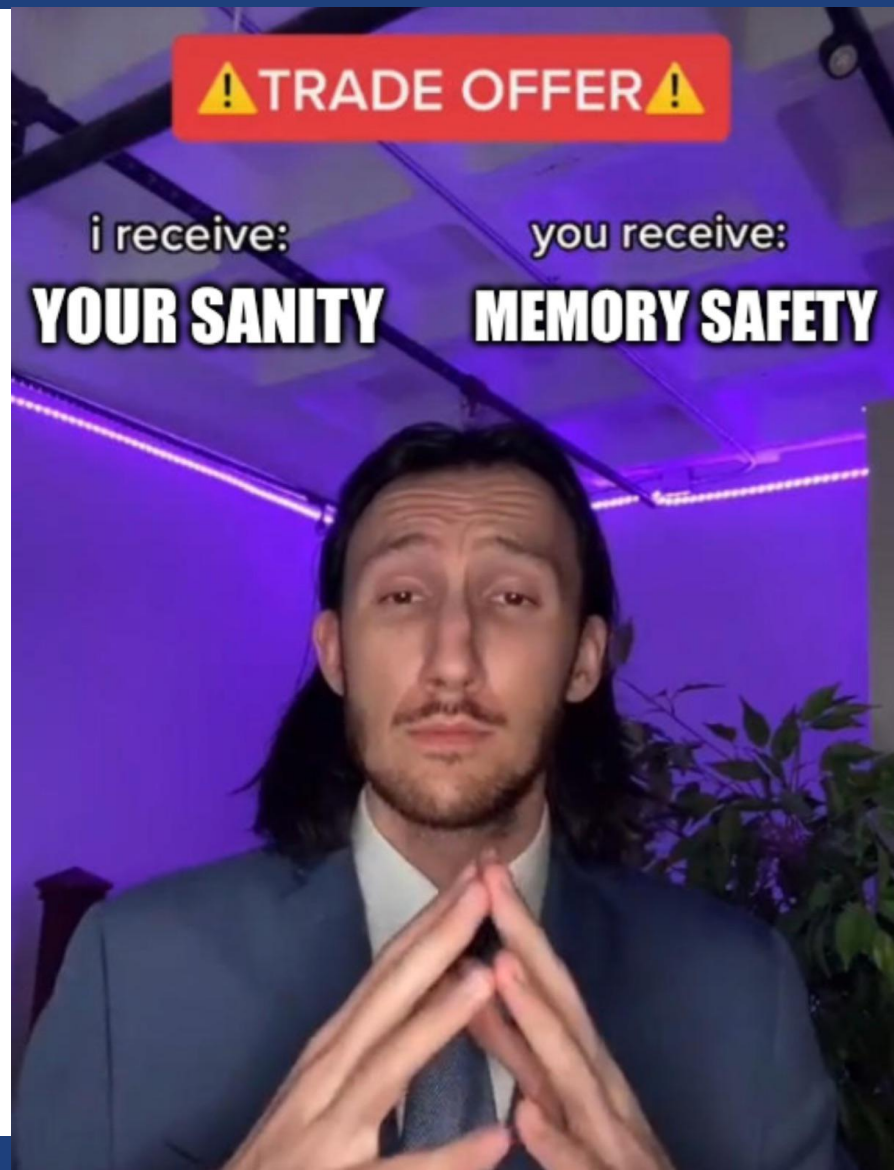
```
long* arr = calloc(10*sizeof(long));
if (arr == NULL) return -1;
// .. do something ..
free(arr);
arr = NULL;
```

- **Solution? Use NULL pointers!**
  - Recall that (1) any attempt to deference NULL causes a segmentation fault
  - And that (2) free(NULL) does nothing and is safe/ignored
  - You should ***always*** NULL a pointer after you have free'd it
    - it's better to have a segfault than to corrupt memory! (MEMORY ERRORS CAN BE SILENT!)

Memory Safety(?) in C

- All sorts of ways to corrupt memory in C

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
  int a[2];
  int* b = malloc(2*sizeof(int));
  int* c;

  b[2] = 5;    // assign past the end of an array
  b[0] += 2;   // assume malloc zeroes out memory
  c = b+3;     // mess up your pointer arithmetic
  free(a);     // free() something not malloc()'ed
  free(b);
  free(b);     // double-free the same block
  b[0] = 5;    // use a free()'d pointer

  // any many more!
  return 0;
}
```



Memory Safety Concerns

- There are several things you need to watch out for when dealing with memory in C (usually dealing with dynamic memory but not always)
  - **Out of bounds (OOB)** errors (arrays/pointers/arithmetic)
    - You've seen this already more or less…
  - **Use after free (UAF)** (dereferencing a dangling pointer that was not NULL'd)
    - Always NULL your pointer after free / do not reuse the pointer/reference/variable
  - **Double free (DF)** (freeing a dangling pointer that was previously already freed)
    - Always NULL your pointer after free / do not reuse the pointer/reference/variable
  - **Invalid free (IF)** (freeing a non-dynamic memory pointer (such as a stack variable))
  - You can't avoid using free; Not freeing allocated variables in time is problematic / antithetical
    - "**Memory leaks**"

Memory leak

- A memory leak happens when code fails to deallocate dynamically allocate memory that will no longer be used

```
// assume we have access to functions FileLen,
// ReadFileIntoBuffer, and NumWordsInString.

int NumWordsInFile(char *filename) {
  char *filebuf = malloc(FileLen(filename)+1);
  if (filebuf == NULL)
    return -1;

  ReadFileIntoBuffer(filename, filebuf);

  // leak! we never free(filebuf) or pass a reference to someone else
  return NumWordsInString(filebuf);
}
```

Implications of a leak?

- A program's virtual memory ***will keep growing***
  - for short-lived programs, this might be OK
  - for long-lived programs, this:
    - might slow down over time (caching issues, swap space, etc…)
    - might exhaust all available memory and crash (OOM)
    - other programs might get starved of memory (silent denial of service)
  - in some cases, you might prefer to leak memory than to corrupt memory with a buggy `free()` *********
    - you're not far enough in programming yet to make this call… ☺

How to avoid memory leaks

PennState

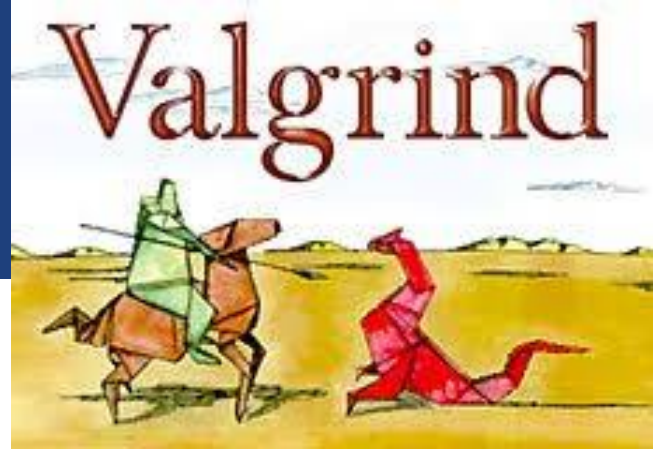
- You can ask a very simple question to avoid memory leaks.

- **Who is responsible for this memory?**
  - **If you are passing a malloc'd pointer back to a caller, then it is the <u>caller's responsibility</u> to free or call a free-like method**

  - **If you are NOT passing a reference to the malloc'd pointer back to the caller, then it is <u>YOUR responsibility</u> to free the memory before your function ends.**

- Not too complex

Finding Memory Safety Issues



- GCC has some static memory safety checks
  - Address sanitization is a fast instrumentation tool within GCC
  - `gcc -fsanitize=address foo.c -o foo`
  - *Will tell you about OOB, UAF, DF, and some others when you run the program!*

- GDB can inspect memory and use watch points to find issues

- Valgrind is a purpose-built debugger for identifying memory leaks and issues
  - More heavyweight than GCC's address sanitizer
  - `valgrind -v --leak-check=full --errors-for-leak-kinds=all ./a.out`

Memory Safety, In Short

PennState

- Always check the return status of malloc/calloc/realloc calls
- Always NULL pointers after they are freed
- Understand who is responsible to free memory
  - Match every malloc with exactly one free!
  - When in doubt, document with comments!
- Free in reverse order of allocation
  - Useful for nested structures (hint hint)
- Use the heap for dynamic or large objects, use the stack for small/temporary

Summary: dynamic memory interface

PennState

```
void *malloc(size_t size);
void free(void *ptr);
void *calloc(size_t nmemb, size_t size);
void *realloc(void *ptr, size_t size);
```

- These are user-space libraries wrapping system calls.
  - You can write one yourself. (And you will in 473)
- Other commercial allocators:
  - dlmalloc (Doug Lea's malloc – you are using its derivative)
  - tcmalloc (thread-caching malloc by Google)
  - jemalloc (Jason Evan's malloc – used by FreeBSD)